EXP 15: Implement Iris Flower Classification using Naive Bayes classifier

CODING:

```
# Iris Flower Classification using Naive Bayes

import math

# Training dataset
data = [
  [5.1, 3.5, "Setosa"],
  [4.9, 3.0, "Setosa"],
  [5.0, 3.4, "Setosa"],
  [6.4, 3.2, "Versicolor"],
  [6.9, 3.1, "Versicolor"],
  [5.5, 2.3, "Versicolor"],
  [6.5, 3.0, "Virginica"],
  [7.0, 3.2, "Virginica"],
  [6.3, 2.9, "Virginica"]
]

# Calculate mean
def mean(values):
  return sum(values) / len(values)

# Calculate variance
def variance(values):
  m = mean(values)
  return sum((x - m) ** 2 for x in values) / len(values)
```

```
# Gaussian probability
def gaussian(x, m, v):
  if v == 0:
    v = 0.0001
  return (1 / math.sqrt(2 * math.pi * v)) * math.exp(
    -((x - m) ** 2) / (2 * v)
  )

# Train Naive Bayes model
classes = ["Setosa", "Versicolor", "Virginica"]
model = {}

for c in classes:
  rows = [row for row in data if row[2] == c]

  model[c] = []

  for i in range(2):
    values = [row[i] for row in rows]
    model[c].append((mean(values), variance(values)))

# Prediction
def predict(sample):
  probabilities = {}

  for c in classes:
    rows = [row for row in data if row[2] == c]
```

```
        probability = len(rows) / len(data)

        for i in range(2):
            m, v = model[c][i]
            probability *= gaussian(sample[i], m, v)

        probabilities[c] = probability

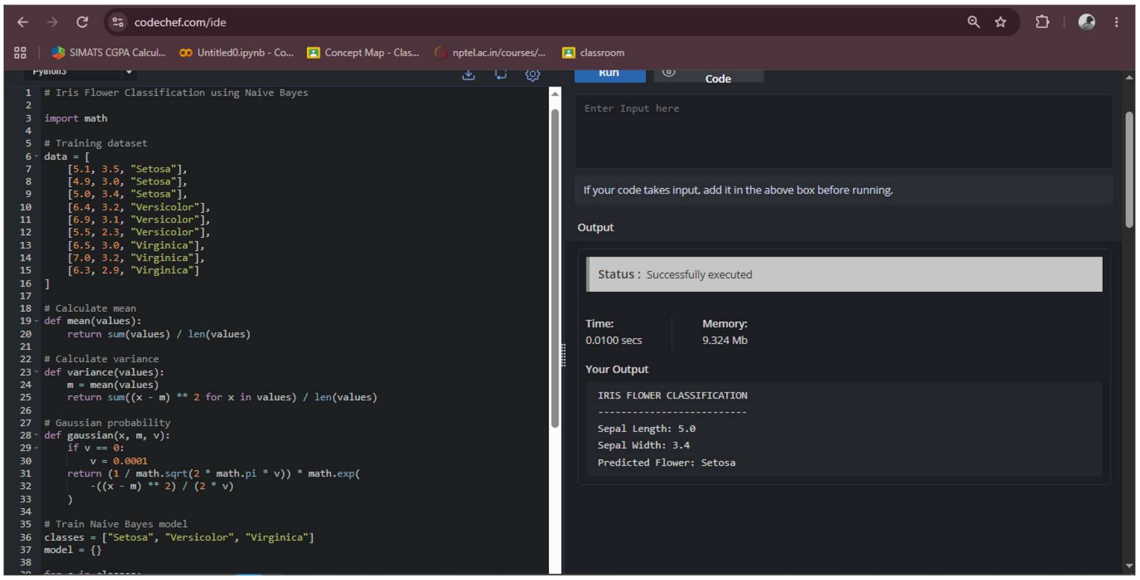
    return max(probabilities, key=probabilities.get)

# Test flower
test = [5.0, 3.4]

result = predict(test)

print("IRIS FLOWER CLASSIFICATION")
print("--------------------------")
print("Sepal Length:", test[0])
print("Sepal Width:", test[1])
print("Predicted Flower:", result)
```

OUTPUT:




```python
# Iris Flower Classification using Naive Bayes

import math

# Training dataset
data = [
    [5.1, 3.5, "Setosa"],
    [4.9, 3.0, "Setosa"],
    [5.0, 3.4, "Setosa"],
    [6.4, 3.2, "Versicolor"],
    [6.9, 3.1, "Versicolor"],
    [5.5, 2.3, "Versicolor"],
    [6.5, 3.0, "Virginica"],
    [7.0, 3.2, "Virginica"],
    [6.3, 2.9, "Virginica"]
]

# Calculate mean
def mean(values):
    return sum(values) / len(values)

# Calculate variance
def variance(values):
    m = mean(values)
    return sum((x - m) ** 2 for x in values) / len(values)

# Gaussian probability
def gaussian(x, m, v):
    if v == 0:
        v = 0.0001
    return (1 / math.sqrt(2 * math.pi * v)) * math.exp(
        -((x - m) ** 2) / (2 * v)
    )

# Train Naive Bayes model
classes = ["Setosa", "Versicolor", "Virginica"]
model = {}
```